CMPSC 465, HW 2 Solutions

Problem 1: Lower Bounds

Part (a)

The problem of finding the median of n elements in the decision-tree model has an information-theoretic lower bound of $\Omega(n)$ comparisons. A simple argument is that to identify the median, an algorithm must at least partition the elements into three groups: those less than the median, the median itself, and those greater than the median. This separation requires a linear number of comparisons in the worst case.

Part (b)

To determine if all n elements in a set are distinct, a comparison-based algorithm must be able to distinguish between all $n!$ possible permutations of the elements. The minimum number of comparisons needed is the height of a binary decision tree with at least $n!$ leaves, which is $\log_2(n!)$. By Stirling's approximation, this is $\Theta(n \log n)$.

Part (c)

When merging two sorted lists of sizes n_1 and n_2, the number of ways to interleave the elements is given by the binomial coefficient $\binom{n_1+n_2}{n_1}$. The information-theoretic lower bound for the number of comparisons is $\log_2 \binom{n_1+n_2}{n_1}$, which simplifies to $\Theta(n_1 + n_2)$.

Problem 2: Overlapping Intervals

A divide-and-conquer algorithm can find the pair of intervals with the maximum overlap in $O(n \log n)$ time. The key steps are:

- Sort the intervals by their left endpoints, which takes $O(n \log n)$ time.
- Recursively find the maximum overlap within the left and right halves of the sorted list.
- Find the maximum overlap between an interval from the left half and one from the right half. This can be done in $O(n)$ time by finding the interval in the left half with the largest right endpoint and then checking its overlap against all intervals in the right half.

The recurrence relation for this algorithm is $T(n) = 2T(n/2) + O(n)$, which solves to $O(n \log n)$ by the Master Theorem.

Problem 3: Peak Elements

A peak element can be found in $\Theta(\log n)$ time using a binary search approach. The algorithm proceeds as follows:

- Examine the middle element of the array, ‘A[mid]‘.
- If ‘A[mid]‘ is a peak, return its index.
- If ‘A[mid+1]‘ is greater than ‘A[mid]‘, a peak must exist in the right subarray, and the search continues there.
- If ‘A[mid-1]‘ is greater than ‘A[mid]‘, a peak must exist in the left subarray, and the search continues there.

This process reduces the problem size by half in each step, leading to a recurrence relation of $T(n) = T(n/2) + O(1)$, which solves to $\Theta(\log n)$.

Problem 4: Median of Medians of Medians

Pivot Guarantee

The "median of medians of medians" pivot, p, guarantees that at least $\frac{2}{9}$ of the elements are less than or equal to p, and at least $\frac{2}{9}$ are greater than or equal to p. This ensures that the size of the larger subarray is at most $\frac{7}{9}n$.

Recurrence Relation

The worst-case running time $T(n)$ is described by the recurrence relation: $T(n) \leq T(n/9) + T(7n/9) + O(n)$. The $T(n/9)$ term accounts for the recursive call to find the pivot, and the $T(7n/9)$ term is for the recursive call on the largest subarray.

Solution

Since the sum of the fractions in the recursive terms $(1/9 + 7/9 = 8/9)$ is less than 1, the recurrence solves to $T(n) = O(n)$, indicating a linear-time performance.

Problem 5: Transpose of a Matrix

Part (a)

The transpose of an $n \times n$ matrix A can be computed recursively by partitioning it into four $n/2 \times n/2$ sub-matrices:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

The transpose A^T is given by:

$$A^T = \begin{bmatrix} A_{11}^T & A_{21}^T \\ A_{12}^T & A_{22}^T \end{bmatrix}$$

The recursive algorithm computes the transposes of the four sub-matrices and then arranges them in the correct positions.

Part (b)

The recurrence relation for the time complexity is $T(n) = 4T(n/2) + O(1)$, as there are four recursive calls on problems of half the size. Using the Master Theorem, this solves to $T(n) = O(n^2)$. This is the same asymptotic time complexity as the standard row-by-column transpose algorithm.